# Believable Characters & Lip-Synchronisation

Emmanuel Ansong
19700916

19700916@lincoln.ac.uk

School of Computer Science

College of Science

University of Lincoln

Submitted in partial fulfilment of the requirements for the
Degree of BSc(Hons) Computer Science

*Supervisor: Mr. Phil Carlisle*

April 2022

Word Count: 7286

# Abstract

The purpose of this project was to produce a believable lip synchronisation system on a 3D animated character, which could be used in a game setting. The project will aim to identify and expand on objects and milestones that were previously set in place during the project's development. Additionally, external literature will be reviewed as well as their contribution to the development cycle. The project was done to see what features could be took from existing lip synchronisation systems as well as their efficiency and effectiveness to the final product. This would all be implemented into Unity's game engine, which was the choice compared with other existing engines.

# Table of Contents

Introduction ..... 4
1.1 Some notes ..... 4
1.2 Background ..... 5
Literature Review ..... 6
2.1 Background ..... 6
2.2 Related Literature ..... 6
Methodology ..... 9
3.1 Project Management and Software Development ..... 9
3.3 Toolsets and Machine Environments ..... 13
Design, Development and Evaluation ..... 16
4.1 Software Development Projects ..... 16
Conclusions ..... 28
Reflective Analysis ..... 30
References ..... 32
Artefact and Media ..... 34

# List of Figures

2.2 Overview of the system in real-time showing how phoneme is recognised from the audio input for the word ‘slowly’. 5

3.2 Gantt chart showing whole timeframe for the project. 13

3.2 Flow of a project using Waterfall and Agile methodologies 13

3.3 Table of performance ratings from Unity and Unreal Engine 16

3.3 Final resulting table of comparison from Unity and Unreal Engine 16

4.1 Example of ARKit blendshape of a person’s mouth moving to the left of the face 19

4.1 Chart of 12 mouth positions used by Disney animators 20

4.1 Daz 3D model in a walking animation pose 23

4.1 Screenshot of project’s user interface 25

4.1 Screenshot of Unity Editor log for viseme events 26

# Chapter 1

# Introduction

## 1.1 Some notes

Believable speech and facial animations usually offer different forms of creating a realistic character to be used in a scene in both the film and gaming industry. Organisations have discovered many alternatives in order to produce a greater and more realistic product, such as the use of algorithms, motion capture and keyframe animation. On the other hand, due to the scale and complexity of some methods, some organisations are limited to smaller methods of speech animation, as in the case of the project, which could be harder to create a believable character model whilst also maintaining realistic speech and facial animation. The purpose of this paper is to aim to recognise and evaluate the specific methods used in the production of the project, to create a realistic and believable 3D character, in which could be viable to be used in a game environment. Furthermore, this paper will explore the main ideas from existing lip-syncing systems, such as their current features and implementations, and will examine and discuss how these systems could be implemented into the project. However, the scope of the project should be able to work with free, open-source libraries which allow viable visemes and phonemes to be extracted from an audio source. In the same way, these visemes and phonemes should be able to drive and complement with the 3D character's animations. The paper will also display and review the specific objectives and milestones as displayed initially in the prior project proposal, as well as their contributions on how they assisted project development.

## 1.2 Background

Lip syncing goes hand in hand with facial animation, which many companies use in order to produce a more believable, realistic product. “A Novel Lip Synchronization Approach for Games and Virtual Environments” by Krejsa & Liarokapis (2010), describes several techniques for creating lip-sync systems with the use of visemes and phonemes. The paper also defines lip sync as “the reinforcement of an illusion that speech is coming from an animated characters mouth”. Developers construct systems which allow for realistic mouth movement with the help of audio inputs, which together, form to assist in adding immersion to characters. Phonemes are described as the smallest unit of sound within a word. (Twinkl.co.uk, 2021) Visemes, on the other hand, are just a collection of these phonemes. A chapter in the Kresja & Liarokapis state in the documentation says that to implement believable lip sync animation, the face must first satisfy certain conditions. The paper also outlines various methods companies will use in order to achieve lip synchronisation. Examples include capturing performance methods of human actors via motion capture, and automatically generating from audio input or phonetic transcripts. Companies are also influenced to hire professional animators to produce facial animations. However, due to the budget and time constraints these methods can be scaled down and duplicated with the use of imported assets, blend shapes and manual animation to create moveable faces.

# Chapter 2

# Literature Review

## 2.1 Background

As stated in the project proposal, the overall context for the project is to create an efficient and effective lip synchronisation system in which produces believable character speech. This system will use free open-source assets and take inspiration from other existing lip sync systems, which will combine animation and algorithmic techniques in order to create a realistic and believable character.

## 2.2 Related Literature

There have been a number of studies which focus on the problem of creating believable speech animation in various industries. Some research insists creating a system which utilizes deep learning networks to create a believable system, such as "A Real-time Phoneme Recognized Network for "Believable" Speech Animation", (Zixiao et al., 2021). The article explains using a set of phonemes and parameters to control the movement of the face. This is complemented alongside a deep learning network which is used to recognize phonemes from an audio input. The paper goes on to describe how the system works, as shown in Figure 1, first by taking the audio input, which is converted to a readable wave spectrum for the system input. Later, the network predicts the phonemes from the audio and maps them to their respective audio offsets. The overall method described in this paper can be used to provide a much more accurate implementation on the recognition of phonemes in contrast with systems which utilise audio frequencies.

Llorach et al. (2016), “Web-based live speech-driven lip-sync” uses their method of a web-based, procedural system which takes live audio input, extracts its phonemes, and creates facial animation using the corresponding blend shapes. These phonemes are later converted into visemes which are checked for every frame. As a result, this method provides almost instantaneous visual results as to compared to deep learning and neural network techniques, which require extensive amounts of time to create a believable system. However, the two papers do suffer some faults, with the limitation being the minimal number of blend shapes used. This hinders their systems in terms of realism since the human face can create more than one shape for the different phonemes and visemes created when speaking. According to Llorach, a reason there is little research on using many blend shapes is due to the fact a large number could prove overwhelming. Furthermore, smaller facial deformations limit the risk of error for a system such as Llorach’s, and for systems such as the one Zixiao has listed, which uses around 400 animations, having to define an individual viseme to a blend shape will be tedious and impractical for the current project.

The results of this paper later conclude that the system is much more reliable compared to deep learning when looking for a fast, but also inaccurate lip sync interface. This complete system, however, cannot be implemented as a standalone foundation for creating a believable system for the project. Although, some features explained in the paper, such as the importance of emotion, are identified as effective additions to create believable animation. This, combined with blend shaped models could be applied simultaneously to create a greater system.

A similar article, Xu et al. (2013), a practical and configurable lip sync method for games describes a similar method to create believable face animation. Their method, which varies from both Llorach’s and Zixiao’s, and requires the use of an animator to 3 drive facial animations, as well as a defined set of phonemes. These animations are then put together to create a final product. The method mentioned does not use any machine learning tools to operate, further allowing for their system

to work with character models that are non-human. This paper also like the previous articles, does not include a way of expressing emotion through its system. The paper explains its method through using a selection of common phonemes, which some are reproduced for facial poses that look similar. This can not only reduce data but reduce the complexity of the system by reducing the number of animations animators need to create.

In addition, the paper talks about having two phases. One phase, called the offline phase constructs animations for the different phonemes, with each phoneme being assigned a respective animation. The runtime phase involves sequence of phonemes taken offline phase which are generated ready for output. Through this, both offline and online input audio can be achieved. Much like Llorach's implementation, this method also allows for different range of models, which is essential as realistic lip sync and facial animation characters become abundant. Further research in this area could involve a better animation to be achieved by simply using multiple sets of phonemes for one animation instead of the two they have tested.

In conclusion, the main point missing in the literature is the consideration into various gestures and facial animations other than the mouth, in the contribution of making a believable character. Generally, facial animation also involves body animations to give the impression of a real character, which the methods labelled in the articles seem to lack. It is important to also consider the expressive faces when working with the facial animation to complement the gestures and speech.

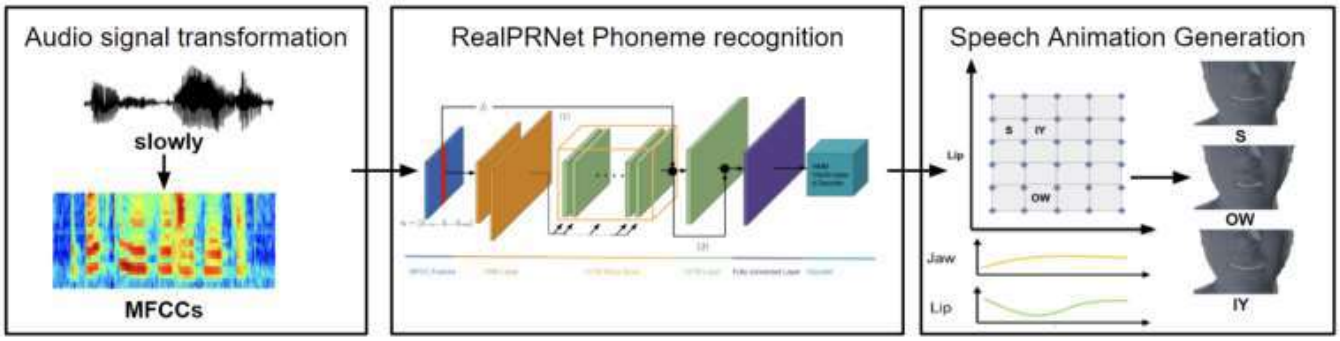



***Fig. 1.*** *Zixiao et al., Overview of the system in real-time showing how phoneme is recognised from the audio input for the word 'slowly'.*

# Chapter 3
# Methodology

The project's predicted aim was to collect research and develop from existing open-source lip sync systems to construct a similar lip-sync system inside a game engine. As a result, several design drivers assisted in development of the system. These drivers include:

• Developing a basic algorithm to take an audio stream input, then output this at visual animation for a character.

• Creating a 3-D lip synchronisation system in Unity

• Research, learn and use an open-source library for speech synthesis and audio input.

• Create an adjustable system for different character models.

## 3.1 Project Management and Software Development

Next is the software development section where we investigate the methodologies used and how the impacted the final product. This section will also cover the schedule that was followed, and how it could be improved for the future.

Software development involves the "Software Development Life Cycle", (SDLC), which is defined as "a structured process that enables the production of high-quality, low-cost software, in the shortest time possible", (Synopsys, 2022). The SDLC allows the planning and tracking of requirements and evaluation of their benefits to the final solution of the project. One methodology involved in the SDLC is the Waterfall methodology.

Following the CRG, the initial design methodology used was Waterfall, as shown in the Gantt Chart in Figure 2. Initially, the Waterfall methodology was the

easiest to understand and was the most preferred method for the project at the time, due to the fact it was a solo project.

Compared to having a team, the Waterfall methodology also seemed the best methodology and could prove difficult to follow when working in a group, since someone could be working on one part of a project, meanwhile another member would focus on a completely different section, which breaks the rules of Waterfall, (Lawal et al, 2021).

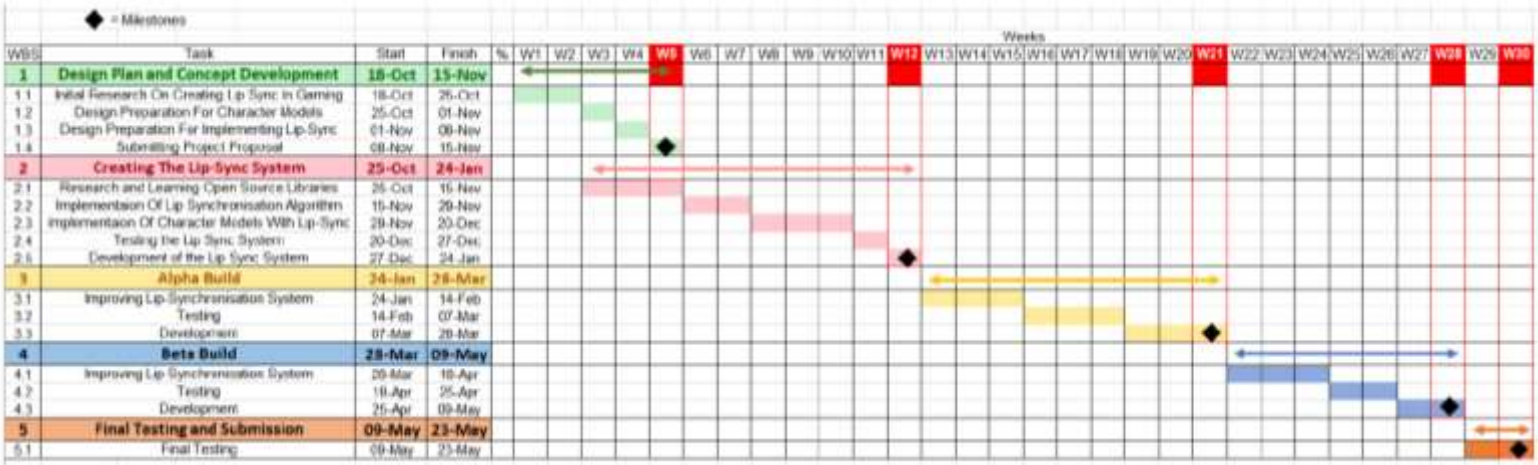
During the development stages of the project, it was decided that it was best to follow a scheduled, structured plan to reach the final objective before submission. The projected outcome would then aim to achieve finishing the project in the specific time period, alongside completing the specified objectives and milestones set in place in the Gantt chart. This was done to guarantee the final product will be as efficient as possible. Furthermore, as compared in "Agile, Waterfall and iterative approach in information technology projects", (Fagarasan et al, 2021), continuing with the Waterfall model allowed the supervisor to give feedback to the current phase of design development and lead the project into the correct direction for the next phase.

Figure 2 showing the Gantt chart of the whole project displays the relevant tasks that were followed, and how these objectives would help the project towards the projected outcome. Each task had been split into separate structures, which considered as phases went through during development. This aided on giving a clear idea on how the scope of the project would be, whilst also giving an idea of when a specific task was expected to be completed, or when objective deadlines were near. The additional milestones that would arise every couple of weeks would act as a further reminder on how the project was staying on track. By doing this, and sticking with the Waterfall model, this would later help in the documentation, evaluation and analysis of the project and the steps taken getting to the deadline. Similarly, by following the Waterfall model, it was easier to be able to keep track of the actual requirements the product needed for submission,

which limited the risk of problems such as: milestones being missed, bad implementation, and feature creep.

As shown in Figure 3, Fagarasan shows the Waterfall method to create a greater likelihood of problems near the end of production in a project, compared to another method such as Agile, which initially, was thought to be impractical considering the circumstances and timescale of the project. Eventually it was decided it was not worth having a product that is missing some features initially established during the design and concept development stage at time of submission.

However, development progressed and milestones we're being met, the initial implementation of the Waterfall model for software development eventually changed to a more Agile methodical approach. This decision was made due to the fact development felt slow using the Waterfall model since the initial plan from the Gantt chart was slightly behind schedule. As soon as concept development was complete, it was immediately time to start the actual project's system. During development it was easier to relay feedback to the supervisor the completed work for that week, discuss improvements, and then adapt the project according to the results. Furthermore, the Agile methodology allowed the project to improve upon its initial project requirements and features in relation to the feedback the supervisor gave, which expanded development ideas and overall made production easier. Similarly, this allowed the project's goals and requirements to stay on track with the projects scope, which further improved development and invited the production of better implemented features. Despite the change of methodologies, this still made the initial development research and features in first phase to be viable. Furthermore, due to the nature of project and it's the timeframe, it was easy to change or add to already existing features.



◆ = Milestones

| WBS | Task | Start | Finish |
|---|---|---|---|
| 1 | **Design Plan and Concept Development** | **18-Oct** | **15-Nov** |
| 1.1 | Initial Research On Creating Lip Sync In Gaming | 18-Oct | 25-Oct |
| 1.2 | Design Preparation For Character Models | 25-Oct | 01-Nov |
| 1.3 | Design Preparation For Implementing Lip-Sync | 01-Nov | 08-Nov |
| 1.4 | Submitting Project Proposal | 08-Nov | 15-Nov |
| 2 | **Creating The Lip-Sync System** | **25-Oct** | **24-Jan** |
| 2.1 | Research and Learning Open Source Libraries | 25-Oct | 15-Nov |
| 2.2 | Implementaion Of Lip Synchronisation Algorithm | 15-Nov | 29-Nov |
| 2.3 | Implementaion Of Character Models With Lip-Sync | 29-Nov | 20-Dec |
| 2.4 | Testing the Lip Sync System | 20-Dec | 27-Dec |
| 2.5 | Development of the Lip Sync System | 27-Dec | 24-Jan |
| 3 | **Alpha Build** | **24-Jan** | **28-Mar** |
| 3.1 | Improving Lip-Synchronisation System | 24-Jan | 14-Feb |
| 3.2 | Testing | 14-Feb | 07-Mar |
| 3.3 | Development | 07-Mar | 28-Mar |
| 4 | **Beta Build** | **28-Mar** | **09-May** |
| 4.1 | Improving Lip Synchronisation System | 28-Mar | 18-Apr |
| 4.2 | Testing | 18-Apr | 25-Apr |
| 4.3 | Development | 25-Apr | 09-May |
| 5 | **Final Testing and Submission** | **09-May** | **23-May** |
| 5.1 | Final Testing | 09-May | 23-May |

***Fig. 2.*** *Original Gantt chart showing whole timeframe for the project.*

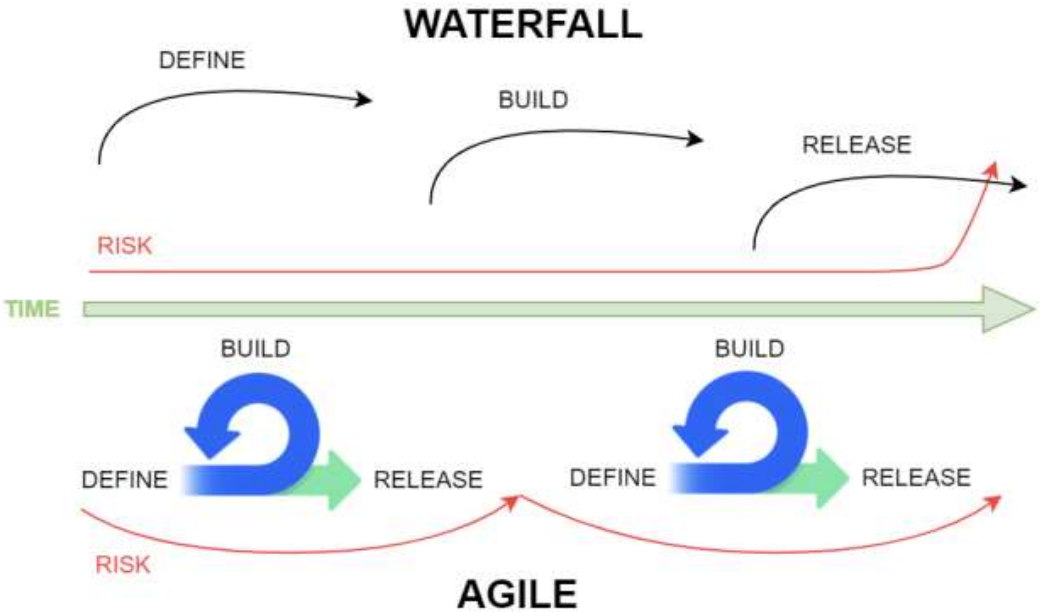



***Fig. 3.*** *Flow of a project using Waterfall and Agile methodologies (Fagarasan et al., 2021)*

## 3.3 Toolsets and Machine Environments

A game engine, as described by Wikipedia, (2018), is “a software framework primarily designed for development of video games, generally including libraries”. Game engines offer the ability to render 3D or 2D assets using computer graphics, allowing for environments to be created by game developers. Alongside graphics, some game engines include opportunity for physics simulation, sound, animation, and much more. Animations, in pair with game engines, can also come in different forms. These include ‘Morphs’, which are used for facial animations, ‘Skeletal’, made to move characters ‘bones’ and virtual armature, and lastly, simple positional and rotational animations. Both morphing and skeletal animations were used to create a realistic model for the project.

In both game engines, Unity and Unreal Engine, the process of creating a lip-synchronisation system on a 3D character is viable, but both engines have their pros and cons. In the paper “A live comparison between Unity and Unreal game engines” by R. Salama and M. Elsayed, Unity’s game engine is described as ‘A multi-functional game engine that can support 2D and 3D graphics through C# scripting’. The alternative, Unreal Engine, was produced by Epic Games, allowing a game engine capable of developing multi-platform, visually scripted games with or without the use of C++. The paper also discusses the Unreal game engine by Epic Games. In particular, Unreal Engine 4, which was the other option when producing the project. Unreal Engine is described as an easily portable system, which would prove beneficial for a project such as this, since it makes it more efficient for production and showcasing. Furthermore, Unreal Engine has far better graphical capabilities over Unity, which at the time seemed the best feat due to the fact the project aimed for realism and believability. However, Unreal Engine, compared to Unity’s engine also has limited 3rd party API’s, which would have hindered development and production if there were undocumented, unsupervised libraries.

As explained in “Comparative analysis of Unity and Unreal Engine efficiency in creating virtual exhibitions of 3D scanned models”, (Ciekanowska et al. 2021), Unity offers some benefits over Unreal Engine, which, at the time, complemented well with the project’s development. The paper discusses the performance and optimisation features both game engines have, along with their impact on various computer systems they tested. Both Figure 4 and Figure 5 are tables taken from the paper. Figure 4 shows developer’s initial ratings of the two game engines from 0 to 5, based on certain criteria, which clearly shows how Unity outperforms Unreal Engine in every aspect of performance and optimisation. In relation to the project, this data proved useful due to the fact the computer the project was developed on was not the best when using Unreal Engine, so Unity proved the better option performance wise. In terms of production, this also proved useful since having a slower running environment would also slow down development and make things tedious. Furthermore, project space was not a factor that needed to be acknowledged, considering the scope of the project, and the fact the goal was not to make a ‘AAA’ system.

At the time was concluded that it was best to use Unity since this was the best game engine for producing the game at the time. Unity’s engine has some advantages over other game engines, such as Unreal Engine, which was the initial choice for creating the project. For example, Unity can easily be built onto multiple platforms, and its interface is well documented. This comes with readily available support for creating 3D games. Similarly, I was most familiar with using Unity, and my skills in C# are far superior compared to its counterpart, C++. As a result, this enabled for faster and more efficient development for the game project, allowing me to have an easier time, and more successful production whilst creating the project.

| Criterion | Unity | | Unreal | |
|---|---|---|---|---|
| | Score | Rating | Score | Rating |
| CPU load | 4.79% | 5 | 14.41% | 3 |
| GPU load | 60.23% | 2 | 81.63% | 1 |
| RAM load | <500 MiB | 2 | >500 MiB | 1 |
| FPS value | 56.38 | 5 | 51 | 4 |
| Overall experience | - | 5 | - | 5 |
| Final project size | <600 MB | 2 | >600 MB | 1 |

***Fig. 4.*** *Table showing results of performance ratings from Unity and Unreal Engine (Ciekanowska et al., 2021)*

| No. | Criterion | Weight | Unity | Unreal |
|---|---|---|---|---|
| 1 | CPU load | 0.1 | 0.5 | 0.3 |
| 2 | GPU load | 0.3 | 0.6 | 0.3 |
| 3 | RAM load | 0.2 | 0.4 | 0.2 |
| 4 | FPS value | 0.25 | 1.25 | 1 |
| 5 | Overall experience | 0.1 | 0.5 | 0.5 |
| 6 | Final project size | 0.05 | 0.1 | 0.05 |
| | Totals: | 1 | 3.35 | 2.35 |

***Fig. 5.*** *Final resulting table of comparison from Unity and Unreal Engine (Ciekanowska et al., 2021)*

# Chapter 4

# Design, Development and Evaluation

## 4.1 Software Development Projects

To begin, it was established that there needed to be justification on why Unity was the choice of game engine over Unreal Engine. In the same way, it was essential to look at the project's initial requirements from the project's scope and make a brainstormed list of those requirements for the future. This was where the Gantt chart and its milestones were introduced to the project, which helped keep the project structured and according to plan.

Afterwards, key points were organised and summarised for documentation of the project plan. These points were later reinstated, and the major points were later highlighted so there was time for describing results and conclusions later. There was also a need to establish a way of making a system work in a way that is easy to use. As a result, there was a lot of research on automation techniques and implementations.

For each supervisor meeting, it was established that having a deliverable piece of material for the supervisor was apparent each week. During discussion of these specific meetings the relevant notes were also acknowledged and documented for future development and reflective analysis. In addition, it was important to acknowledge and outline the risks of meeting those deliverable expectations, which led to making notes of countermeasures to mitigate risks and ensure the project moved accordingly. Some examples include setting time aside for development or doing efficient and proper research.

It was also important to take note of the current resources that were exposed at the time, such as the timeframe of the project and the fact it was a solo project. As a result, it was noted that boundaries had to be set, as stated in the initial

project proposal and Gantt Chart. For example, having end dates and being limited to free open sources assets. In the same way physical resources also had to be acknowledged. One in particular was the fact the project was running on a low-spec computer, so optimisation and other graphical features had to be addressed.

There was also a need to research existing systems to ensure the correct systems were being researched and were related to project. There was caution not look for irrelevant implementations such as neural networks rather mainly facial recognition and lip sync systems. For the specified research, their algorithms were also noted alongside how they implemented animations.

The initial design idea was to have a single character model, in which would be able to be animated and brought to life by lip syncing. This model would be accompanied by a scene and be able to animate that character according to what the character is made to say in a text box.

However, straying away from this design idea slightly, of the first builds made was a facial tracking and recognition system using OpenCV. Initially, the plan was to have a camera setup, which would be able to match the movements of a face and implement those same movements onto a 3D Model, however this design idea was quickly scrapped after there was troubles getting Unity to work with OpenCV. Furthermore, my computers camera was having trouble getting a clear image, which inhibited its ability to recognise faces properly.

Initial systems that were looked at during development phase include “SALSA LipSync Suite”, in which they describe as “system for breathing life into game or application characters/avatars”, (General Overview - SALSA LipSync Suite v2, 2022). One feature which inspired design development in the project is their use of randomised eye tracking. This feature also helped develop the project’s own character model where randomised brow movement would go alongside the character’s lip animation. Looking at their ‘how-to’ guides and documented videos also allowed me to grab ideas for my own project.

There was also a chance to look at Unreal Engine's Metahuman feature, which is a hyper realistic, high-fidelity system which allows users to create and animate humanoid characters. Unfortunately, the hardware the project was running on could not run properly so this showcase was brief. However, after viewing tutorials and documentations there were discussions on how Unreal Engine uses facial rig mechanisms and animation sequences to produce a realistic human, which helped in the production of the project. Furthermore, after looking at their character models I took noticed on how they use the muscles to make up a facial expression instead of only the lips moving. They made it clear that to create realistic movement there should be a collection of muscles moving such as the chin, cheeks and jaw.

Apple's ARKit is a software which allows users to create virtual environments using Augmented Reality (AR). Augmented Reality is "an additional 2D or 3D experience for the user with adds elements from a devices camera view", (Apple Developer Documentation, 2022). AR comes with various features such as motion tracking and object scanning making it a preferable and completely viable method for lip syncing systems by developers. ARKit's XR Plugin for Unity includes an additional dictionary of facial features known as blendshapes, which provide various facial expressions and facial animations based on the recording from the user's camera. The ARKIT XR Plugin comes with 52 different blendshapes which all coexist to create realistic and believable face movements. This is another feature which helped inspire the final design in the project.

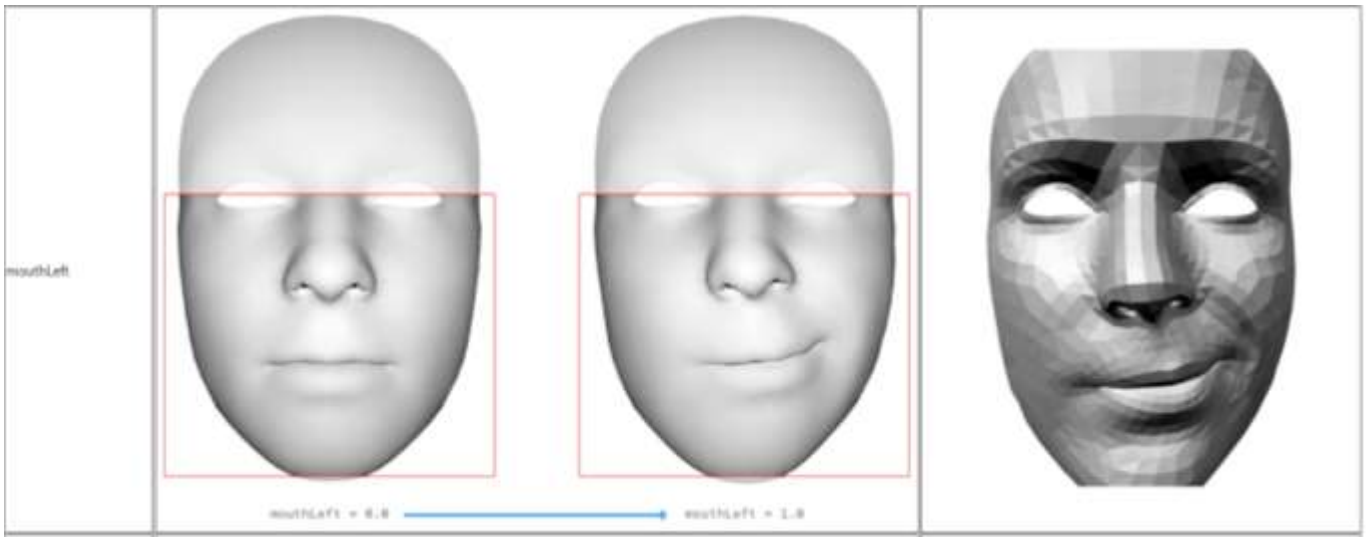



***Fig. 6.*** *Example of ARKit blendshape of a person's mouth moving to the left of the face (ARKit Blendshapes, 2022)*

CryEngine's documentation was another helpful source of information that was able to be looked at during the design phase. The documents explain the difference between visemes and phonemes meanwhile also explaining how they relate to one another. Another beneficial piece of information about visemes mentioned in this documentation is the description on how animators at Disney create lip syncing. They acknowledged using a vast number of phonemes was unnecessary, and they were able to create believable mouth movements with as little as 12 mouth positions. This later helped with the starting point in the project, during development of the pre-alpha build in which there would be a showcase in the supervisor meetings.

***Fig. 7.*** *Chart of 12 mouth positions used by Disney animators*
*(Phonemes and Visemes - CRYENGINE 3 Manual - Documentation, 2022)*

Another system that helped influence the design phase in the project is a company called Resemble AI. Resemble's website introduced ways of generating AI speech and implementing these voices into Unity's Engine. I experimented with the system and took notice of how they integrated a number of features into Unity. Resemble's character model included hand gestures, one facial expression and the ability for the model to look at the camera. Looking at their code, I saw how they went about blending and interpolating between phonemes whilst also taking note of the number of phonemes they used. Finally, they explained how they allowed the user to add emotion to their Text-To-Speech feature, which added realism to the character.

A lot of the program and code used in the unity project takes inspiration from Microsoft Azure's Speech service documentation, which helped contribute towards the making of lip sync in the project. The Azure speech service offers a variety of resources, ranging from Text-To-Speech to keyword recognition services for developers to use in their own programs. It also allows accessibility for various languages and regions.

For the project, Microsoft Azure's Text-To-Speech (TTS) was used, (also known as speech synthesis), to work with the project and Unity's game engine. TTS, as explained on the Microsoft documentation, is the ability to "convert text into humanlike synthesised speech". For the system to work with the project itself, a Microsoft Azure account is required, which will provide users with an account subscription and a key to which can be input the program. This, alongside the specified region will let the project run accordingly. Azure's TTS comes with 'viseme synthesiser' which allows the production of lip synchronisation in the program. This viseme synthesis makes up the main bulk of the program code, utilising audio files to create interactive and believable characters. The TTS viseme synthesiser takes Waveform Audio File (WAV) as input and is able to extract the words from the audio and output its visemes along with their timestamps to the console.

During development, there were also some issues with getting the converted WAV files to get in sync with the characters lip movement. In order to fix this, Azure's TTS can be paired with Speech Synthesis Markup Language (SSML), in which had to be implemented with the Unity project. The SSML code could be finely tuned to compensate for pauses and breaks in the original WAV file audio, which suppressed the delay between the characters lips and the audio.

In the project various programming techniques were considered. Throughout the program are several methods being called which made coding more efficient since I didn't have to re-write the code again. Other object-oriented programming features in the code include Abstraction, which helps hide unwanted, inessential data from the user, Encapsulation, through access modifiers and lastly Polymorphism. Variables have also appropriately named so they would be easier to refer to and easier for other programmers reviewing the code. As for error handling, due to the nature of the program there weren't many errors I needed to handle, except for when implementing the Microsoft Azure to communicate with their servers.

The 3D character model used in the project comes from an application called ‘Daz 3D’, which allows 3D models to be created and exported as Adaptable File Formats, or “fbx”. Initially the idea was to use free Unity assets, but unfortunately there was difficulty getting correct blendshapes along with compatible animations to work. The application allows compatibility with Unity’s engine, all for free, which made development easier, compared with other 3D modelling applications where I had to conduct additional settings to get a model working, which was tedious.

***Fig. 8.*** *Daz 3D model in a walking animation pose*

Next, are the animations, which were taken from a website called Mixamo, which offers free animations which could be used with the 3D Model from Daz3D. The specific animations were chosen for the project to make the lip sync seem more realistic along with additional hand gestures and body movement.

The project uses Unity's timeline interface to implement animations with lip-sync. Since the characters animations are predefined, the project was made so the animations work well alongside the audio files. A better alternative would be to have procedural animations, which might seem more realistic and adaptable if I was to change the audio.

During development there were various discussions on how the 3D model's face could be improved to make the character speech seem more believable. One method noticed was through facial expressions. As a result, research was done on Facial Action Coding Systems (FACS), which are "a set of facial muscle movements that correspond to a displayed emotion" (Farnsworth, 2019). These muscle movements are essential in creating a believable character face since they replicate the movement of an actual human face, which is the goal I was trying to achieve in the project. In addition, many professional companies such as Disney and Apple use FACS for animators and facial recognition, further reinforcing its benefits in the project.

With the help of "Enhancing the explanatory power of usability heuristics" (Nielsen, J., 2020), there was a way of developing and discussing the positive and negative aspects when 3D lip-sync systems. In terms of UI interaction, coming up with different types of interaction can be either more or less useful for different systems. When considering the placement of the GUI, it was important to put the UI in a position where the user could still have an interactive experience, whilst being able to modify and experience the system. An additional requirement was also acknowledged at this time, by taking notice of having an easily usable system. This addition also made the project design look cleaner and made development and testing easier in the long run.

***Fig. 9.*** *Screenshot of project's user interface*

Firstly, playtests were done throughout the development of the project which allowed any bugs to be noticed and dealt with accordingly. In the same way, feedback from my supervisor helped identify features which needed to be changed, however, this feedback could not highlight many faults but rather only development ideas. Playtesting, along with logging helped assist in making sure everything was working as intended. An example includes making sure the synthesiser outputted the correct words from the audio file input (Figure 10).

Load and Compatibility tests were also done, although these forms of testing were looked at briefly. Considering the scope of the project and its scale, adapting the project to run on lesser hardware seemed useless and I concluded it was unlikely users would have difficulty running the project on their personal devices. On the other hand, if the project was to be run on another device such as a mobile, there may be difficulties with the UI and resolutions.

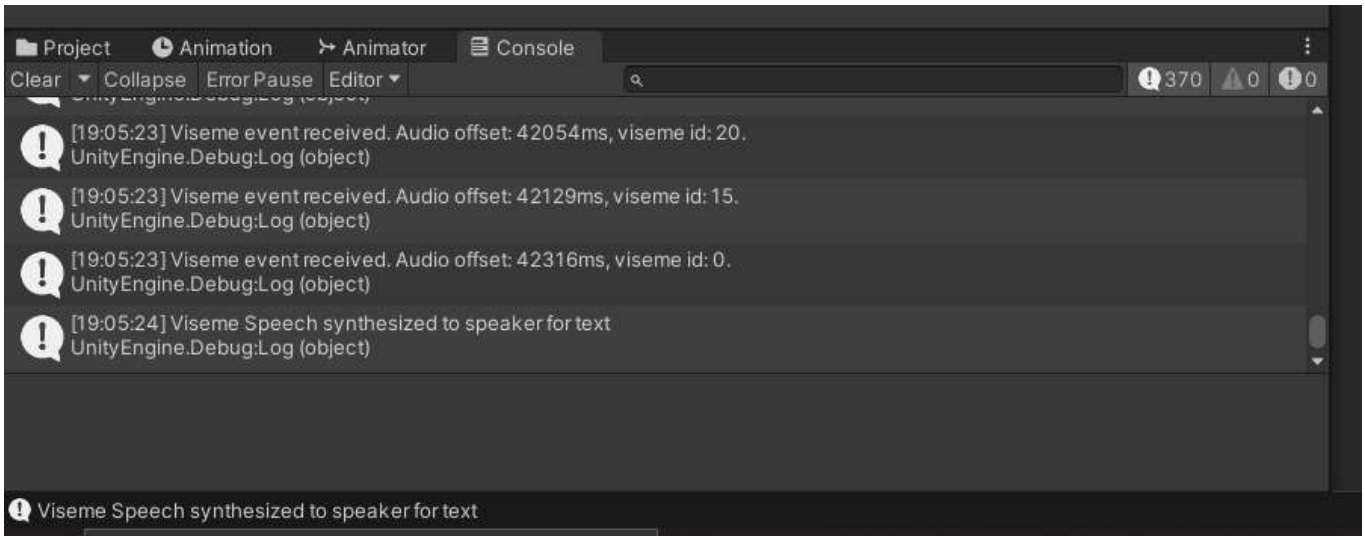



***Fig. 10.*** *Screenshot of Unity Editor log for viseme events*

# Chapter 5

# Conclusions

To conclude, the variety of research questions originally proposed in the introduction could help determine whether the project was efficient and effective in finding the problem's solution.

For the question of 'How do the methods used in production help create a 3D character?', we must first initialise the list of those methods. These include animating the character whilst audio is being played to give a realistic feel. Next, using phonemes and visemes to replicate human speech, and using these two together to create a believable character, and lastly, using an audio script to create a dialogue for the created character to use. These listed methods all contribute towards the project in some way and help in giving life to the 3D character model.

Existing systems were also used and experimented with in the projects final stages. Using features such as randomly generated facial movements from 'LipSync SALSA's' implementation help the 3D character in becoming more believable. However, the project's version of this feature could be altered slightly, since it currently feels forced and convulsive. Similarly, the existing lip-sync systems such as ARKit's use of blendshapes have also been considered when creating the 3D character, which help in creating a conscious-like character with standard facial features and emotions. Finally, CryEngine and Disney's documentation were also implemented throughout the whole project, by giving guidance on how one should go about creating the phonemes and visemes. These documents inspired ideas on how the visemes should look and how they should interjoin with one another.

As for the question of 'How are free, open-source assets and libraries used to create a believable 3D character?', the free, downloadable asset from Daz 3D Studio helps answer this. The model can easily be imported and exported into the project which offers efficiency. On the other hand, Daz 3D models compared to other studios could prove quite lacklustre due to the face being smooth, which does not bring out the facial features and bone structures as much as other models. As a result, this can limit the believability of the characters speech in some cases. Another example of an open-source library used is the Microsoft Azure's Speech Service system, which allows the project to illustrate facial pose events in the game engine.

Finally, to demonstrate how the initial project milestones have contributed to the project's development, a basic algorithm was created with the help of the Microsoft Azure library as stated before. The library and algorithm combined makes the project capable of accurately taking an audio input stream and outputting a lip animation. However, this does not completely satisfy the question, since the audio is not directly made into visual animation. Instead, the audio is first converted into a text format, and from there, visemes are extracted from the text and a visual animation is produced. Similarly, from the extracted audio, there is not any real emotion taken from it, so this also limits the project in terms of efficiency and effectiveness.

Also, Microsoft Azure's relevant library was researched and was found to be a relatable open-source library for speech synthesis, further answering the question on how the project milestones contributed towards project development. The library conveniently offers options for a variety of different languages which can be useful in some cases. Furthermore, the library also offers an option for creating a natural AI voice, which could be replaced instead of audio input if need be. The library's use in the project, however, could be improved in a number of ways. For example, exploring the full capability of the library by editing the SSML code to allow for customisable voices. This feature could easily be implemented into the project could prove beneficial in creating a more realistic character. This, combined with increased emphasis from the customised AI voice, could even replace the original audio input set place in the project.

For the 3D character model, the project does offer an adjustable method of substituting the character for another model. Although, this substitution may need some change in code since the project was built specifically for the 3D Daz Model used, which is a male character. However, this is easily changeable as it's only a few minor variable name changes.

# Chapter 6

# Reflective Analysis

Overall, throughout the course project I think it was successful, however, I do believe the project could have been even better because of a couple reasons.

Given 20:20 hindsight, I believe the project could have been included with a better character model, which would have made the lip sync system stand out more. In addition, a greater timescale and later deadline for the project would have introduced the opportunity for more previews, since I think currently the project doesn't showcase itself as well as it could do. Furthermore, I think if I was able to get a better device and run Unreal Engine, I could have made a more aesthetically pleasing system, and maybe have included additional features I could not originally implement in Unity.

As for what went well in the project, I think I was able to complete the project to a good standard given the time limit I had. I also think the weekly meetings which me and my supervisor had helped drastically since it helped me keep focus on the project's goal and kept the project on track. In addition, I was able to respond well to my supervisor's feedback. I was also able to follow and reach the project's deadlines efficiently, and any time I lost I would quickly make up during the stages of development. This by itself gave me additional confidence throughout the final durations in production.

On the other hand, I do also think there were some moments in development which could have dramatically improved the final design.

During production, I unfortunately lost some progress in the project by losing work I had done a couple weeks into development. This made me unmotivated, due to the lost time and effort I put in at the time. This silly mistake was also a result of purely not having backed up stages and having unsaved work. Luckily, I was able to recover a little bit and was able to continue my work, but from a prior stage in development. With this fault, I think the project could have been even greater if this issue didn't occur. As a result, in order to prevent from happening again, I made sure to back up work with version control tools like GitHub. This change also made production easier, since I could refer back and forth between old and new work, which helped massively when dealing with bugs and minor issues in development.

Similarly, I think my overall organisation skills during development were poor, which hindered my progress. I think the ways I went about documenting during development was flawed, since everything was all over the place, which made it difficult when referring back to previous stages, purely because of disorganisation. In the same way, I also do believe the final design and the features which make up the project is still somewhat lacking in some areas and in terms of features.

If I had more time, and less project constraints, I think adding a bit more of these features could have made my project stand out from other systems just a bit more. For example, a better-looking model, as mentioned before. Additionally, if I had spent more time during the development research phase of production, I think this would also improve the final design stages near the end of the development lifetime. This might have also improved how refined the project's system would be, by adding some originality to the project compared to other systems.

In terms of coding, having a bit stricter coding/commenting style would have helped for future reference when I looked back at project, since I only really detailed my code with comments after everything was complete.

Finally, as mentioned, if the project contained more previews and possibly a pre-set function for the character's emotions, this would have best option in terms of showcasing.

# References


Aad, Georges et al. (2012). ‘Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC’. In: *Physics Letters B* 716.1, pp. 1–29 (cit. on p. 2).

Lawal, A. and Ogbu, R.C., 2021. A Comparative Analysis of Agile And Waterfall Software Development Methodologies. BAKOLORI JOURNAL OF GENERAL STUDIES, 11(2), pp.1-2.

Fagarasan, C., Popa, O., Pisla, A. and Cristea, C., 2021, August. Agile, waterfall and iterative approach in information technology projects. In IOP Conference Series: Materials Science and Engineering (Vol. 1169, No. 1, p. 012025). IOP Publishing.

Llorach, G., Evans, A., Blat, J., Grimm, G. and Hohmann, V., 2016, September. Web-based live speech-driven lip-sync. In 2016 8th International Conference on Games and Virtual Worlds for Serious Applications (VS-GAMES) (pp. 1-4). IEEE.

What Is the Software Development Life Cycle (SDLC) and How Does It Work? | Synopsys (2022). Available at: https://www.synopsys.com/glossary/what-is-sdlc.html (Accessed: 21 May 2022).

ALTVATER, A., 2020. What Is SDLC? Understand the Software Development Life Cycle. [online] Stackify. Available at: <https://stackify.com/what-is-sdlc/> [Accessed 19 May 2022].

Krejsa, J. and Liarokapis, F., 2021, August. A Novel Lip Synchronization Approach for Games and Virtual Environments. In 2021 IEEE Conference on Games (CoG) (pp. 1-9). IEEE.

Yu, Z., Wang, H. and Ren, J., 2021. RealPRNet: A Real-time Phoneme Recognized Network for “Believable" Speech Animation. IEEE Internet of Things Journal.

Xu, Y., Feng, A.W., Marsella, S. and Shapiro, A., 2013. A practical and configurable lip sync method for games. In Proceedings of Motion on Games (pp. 131-140). Author, F., Author, S. and Author, T. (2010): Book title. 2nd edition. Publisher, Location.

Ciekanowska, A., Kiszczak-Gliński, A. and Dziedzic, K., 2021. Comparative analysis of Unity and Unreal Engine efficiency in creating virtual exhibitions of 3D scanned models. Journal of Computer Sciences Institute, 20, pp.247-253.

Game engine - Wikipedia (2018). Available at: https://en.wikipedia.org/wiki/Game_engine (Accessed: 15 May 2022).

Twinkl.co.uk. 2021. [online] Available at: https://www.twinkl.co.uk/teaching-wiki/pho-neme [Accessed 17 November 2021].

Farnsworth, B., 2019. Facial Action Coding System (FACS) - A Visual Guidebook. [online] Imotions. Available at: <https://imotions.com/blog/facial-action-coding-system/> [Accessed 17 May 2022].

Šmíd, A., 2017. Comparison of unity and unreal engine. Czech Technical University in Prague, pp.41-61.

Wu, Y., Chen, Y., Xie, X., Yu, B., Fan, C. and Ma, L., 2020, September. Regression Testing of Massively Multiplayer Online Role-Playing Games. In 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME) (pp. 692-696). IEEE.

Phonemes and Visemes - CRYENGINE 3 Manual - Documentation (2022). Available at: https://docs.cryengine.com/display/SDKDOC2/Phonemes+and+Visemes (Accessed: 19 May 2022).

Apple Developer Documentation (2022). Available at: https://developer.apple.com/documentation/arkit (Accessed: 16 May 2022).

ARKit Blendshapes (2022). Available at: https://arkit-face-blendshapes.com/ (Accessed: 19 May 2022).

General Overview - SALSA LipSync Suite v2 (2022). Available at: https://crazyminnowstudio.com/docs/salsa-lip-sync/modules/overview/#:~:text=SALSA%20(Simple%20Automated%20LipSync%20Aproximation,lot%20of%20time%20or%20effort. (Accessed: 19 May 2022).

Nielsen, J. (2020) 10 Usability Heuristics for User Interface Design. Available from https://www.nngroup.com/articles/ten-usability-heuristics/ [accessed 1 February 2022].

# Artefact and Media

Demonstration video link: https://youtu.be/pZKcV5evtjM

Link to the project via GitHub: https://github.com/emmanuelansong/LipSync-Project